



Lab Manual – 7

Code/Course: 23CSE312/Distributed Systems

Topic: MPI Cluster

[CO05] [BTL-3]

Student Name: G T e j a s w i n i

Roll No.: A M . S C . U 4 C S E 2 3 2 1 9

Batch/Section: C S E C

Date: 9/5/25

Objective

To create a small Linux-based cluster using MPICH and execute parallel programs using MPI, including a basic Hello World program and a parallel summation program.

1 Part A: Cluster Creation and MPI Setup

You will need two or three Linux systems on the same network. Alternatively, you may create two or three virtual machines or AWS instances running Linux, provided they can communicate with each other.

The MPI implementation suggested for this exercise is MPICH. Using this, the systems can be configured as a small Beowulf cluster. Once the cluster is configured, execute a Hello World MPI program on the cluster.

1.1 System Requirements

- 2–3 Linux machines, virtual machines, or AWS EC2 instances
- All machines should be on the same network
- All machines should support SSH communication
- Passwordless SSH access should be configured between nodes

1.2 Install MPICH on All Nodes

Run the following commands on each Linux system:

```
sudo apt update
sudo apt install mpich -y
```

1.3 Setup Passwordless SSH Access

On the master node, generate an SSH key:

```
ssh-keygen -t rsa
ssh-copy-id user@node1.ip
ssh-copy-id user@node2.ip
```

Test the SSH connection using:

```
ssh user@node1_ip
```

1.4 Configure the Hosts File

On the master node, create a hosts file containing the IP addresses of all nodes:

```
192.168.1.10
192.168.1.11
192.168.1.12
```

2 MPI Hello World Program

2.1 Source Code: hello.c

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(NULL, NULL);

    int world_size, world_rank;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    printf("Hello-from-processor-%d-out-of-%d-processors\n",
           world_rank, world_size);

    MPI_Finalize();
    return 0;
}
```

2.2 Compilation

```
mpicc -o hello hello.c
```

2.3 Execution on Cluster

```
mpirun -np 3 -hostfile hosts ./hello
```

3 Part B: Parallel Addition of Numbers

Once the cluster has been set up, implement a parallel program to add a large list of numbers stored in a data file.

In designing the parallel solution, the following tasks must be performed:

1. Separate the task into divisions and assign each process one division.
2. Transfer data between processes.
3. Form the final output using the result obtained from each process.

Additionally, keep the following ready for demonstration:

- The .c source files
- The data file used
- Screenshots of terminal execution

```
users12@users12-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7$ mkdir mpi_lab
users12@users12-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7$ cd mpi_lab
users12@users12-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ nano hello.c
users12@users12-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ mpicc -o hello hello.c
users12@users12-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$
users12@users12-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$
users12@users12-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ mpirun -np 3 ./hello
Hello from processor 0 out of 3 processors
Hello from processor 1 out of 3 processors
Hello from processor 2 out of 3 processors
```

3.1 ut Data File: data.txt

```
1 2 3 4 5 6 7 8 9 10
```

3.2 Source Code: sum.c

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int main(int argc, char** argv) {
    int rank, size, n = 10;
    int data[MAX], local_sum = 0, total_sum = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk = n / size;

    if (rank == 0) {
        FILE *f = fopen("data.txt", "r");
        for (int i = 0; i < n; i++) {
            fscanf(f, "%d", &data[i]);
        }
        fclose(f);
    }

    int* sub_data = (int*) malloc(chunk * sizeof(int));

    MPI_Scatter(data, chunk, MPI_INT, sub_data, chunk, MPI_INT, 0,
                MPI_COMM_WORLD);
```

```
for (int i = 0; i < chunk; i++) {  
    local_sum += sub_data[i];  
}
```

```
MPI_Reduce(&local_sum, &total_sum, 1, MPI_INT, MPI_SUM, 0,  
          MPLCOMM_WORLD);
```

```
if (rank == 0) {  
    printf("Total sum = %d\n", total_sum);  
}
```

```
aditya@aditya-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ nano data.txt  
aditya@aditya-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ nano sum.c  
aditya@aditya-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ mpicc -o sum sum.c  
aditya@aditya-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ mpirun -np 2 ./sum  
aditya@aditya-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$ mpirun -np 2 ./sum  
Total sum = 55  
aditya@aditya-Inspiron-15-3525:~/Desktop/school/Distributed/labsheet_7/mpi_lab$
```

```
    }  
  
    MPI_Finalize ();  
    return 0;  
}
```

3.3 Compilation

```
mpicc -o sum sum.c
```

3.4 Execution

```
mpirun -np 2 -hostfile hosts ./sum
```

4 Optional Enhancements

The program may be extended further using the following MPI operations:

- `MPLGather()` for collecting partial results
- `MPLBcast()` for broadcasting data to all processes

Conclusion

This experiment demonstrates the creation of a simple MPI-based cluster and the execution of parallel applications using MPICH. It illustrates both basic inter-process coordination through a Hello World program and distributed computation through parallel summation.